A Sweeter Lesson? A Second Look at Big AI for Software Engineering

Anonymous Author(s)
Anonymous Institution

***Abstract*—Sutton's bitter lesson says that general methods riding ever-cheaper computation ultimately beat approaches built from human knowledge. Written in 2019, before GPT-3, the essay predicted the scaling era we now live in. This paper rereads it from inside software engineering (SE) and reports a sweeter lesson. All four of Sutton's motivating examples (chess, Go, speech, vision) enjoy fast, cheap, reliable feedback, so computation can substitute for knowledge. Much of SE analytics has no such oracle: labels cost hours of expert time or whole test-suite runs. There, the bitter lesson's preconditions fail, and radical simplicity wins. Across 127 multi-objective SE optimization tasks, an active learner that labels just 50 examples, then checks five, reaches 85% of the best known result, via trees of four to nine attributes, in two hours on one laptop. Yet in a survey of 229 papers on LLMs in SE, only 5% compared against anything simpler. We propose an agenda to fix that: baseline the simple; ask, at review time, "compared to what simpler?"; and teach easier AI first, scaling up only when the simple fails.**

***Index Terms*—active learning, search-based software engineering, green AI, empirical standards**

I. Introduction

In March 2019, before GPT-3 existed, Sutton published a short essay called *The Bitter Lesson* [1]. Figure 1 reprints it in full. Its claim: general methods that leverage computation beat methods built from human knowledge; always, eventually, and by a large margin. The essay was prescient. It described the scaling era before that era arrived, and its unlimited-CPU perspective now looms over most current thinking about AI, including AI for software engineering (SE). This paper is a second look at that lesson, from inside SE. We will argue that, for much of this field, there is a sweeter lesson going unlearned.

The cost of not looking compounds. Lehman's second law warns that a system grows ever more complex unless work is done to reduce it [2]. AI-for-SE is now such a system: models, training sets, energy budgets, and invoices all grow by orders of magnitude, and little work pushes the other way.

This paper pushes the other way. We ask two questions. First: how much of AI-for-SE could be radically simpler? Recent evidence, summarized in §III, says much of it [3]. Second: if simple works so well, why do so few papers check? In a survey of 229 papers on LLMs in SE, only 5% baselined against anything simpler [4]. We call that a methodological error. We also think it is a fixable one.

One caveat, stated early: generative tasks (translation, code summarization, natural-language interfaces) genuinely need large models. But many SE tasks (configuration, tuning, defect prediction, testing, cost estimation) have far simpler solutions, and this paper is about those.

The experiments cited here appeared, or will appear, in prior papers by Ganguly, Rayegan, Senthilkumar, Srinivasan, Menzies, and colleagues [3], [5], [6], [7], [4]. What is new here is the rereading of Sutton, the synthesis, and the agenda: §II argues the why, §III the how, §IV diagnoses why not, and §V plans the full-length empirical result.

II. Why Easier AI?

Before the economics, reread the blue paragraphs of Figure 1. Sutton offered four examples: chess, Go, speech recognition, and computer vision. Note what they share. A chess or Go program scores a candidate move by playing millions of games against itself; speech and vision systems train against millions of labeled utterances and images. In every case, feedback is immediate, cheap, and incorruptible, so computation can substitute for knowledge without limit. That is the essay's silent premise. We do not dispute Sutton's history. We dispute the transfer: where feedback is slow, dear, or scarce (§III), scale loses its lever, and the lesson turns sweeter. Start with money. Thirty years ago, Wirth complained that software gets slower faster than hardware gets faster [8]. Even as the price per token falls, Jevons' paradox applies: cheaper inference means more inference, and total token consumption has grown by orders of magnitude since 2023. Bain & Company forecasts that by 2030, the AI industry must find $800 billion per year in new revenue to fund its own compute build-out; capital expenditure was near $725 billion in 2026 and is projected to pass $1 trillion in 2027 [9]. Someone must pay for all this, and researchers who build on rented frontier models should ask what happens when the subsidies stop.

Money is one worry. Trust is another. A model small enough to read is a model that can be audited, repaired, and defended in front of a regulator. Rudin argues that high-stakes decisions need interpretable models, not post-hoc explanations of black boxes [10]. Bender et al. ask, of language models, "can they be too big?" [11]. For much of SE analytics, the honest answer is yes. In one study, 52% of ChatGPT answers to StackOverflow questions were partly incorrect, and readers overlooked those errors 39% of the time [12]. Scale the generation and you scale the verification queue [13].

Some domains do not get a choice. The Green AI literature documents the energy and carbon cost of large models [14], [15]. Beyond that: much of the global south lacks American-scale infrastructure; community colleges and K–12 classrooms

THE BITTER LESSON
Rich Sutton, March 13, 2019

The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin. The ultimate reason for this is Moore's law, or rather its generalization of continued exponentially falling cost per unit of computation. Most AI research has been conducted as if the computation available to the agent were constant (in which case leveraging human knowledge would be one of the only ways to improve performance) but, over a slightly longer time than a typical research project, massively more computation inevitably becomes available. Seeking an improvement that makes a difference in the shorter term, researchers seek to leverage their human knowledge of the domain, but the only thing that matters in the long run is the leveraging of computation. These two need not run counter to each other, but in practice they tend to. Time spent on one is time not spent on the other. There are psychological commitments to investment in one approach or the other. And the human-knowledge approach tends to complicate methods in ways that make them less suited to taking advantage of general methods leveraging computation. There were many examples of AI researchers' belated learning of this bitter lesson, and it is instructive to review some of the most prominent.

In computer chess, the methods that defeated the world champion, Kasparov, in 1997, were based on massive, deep search. At the time, this was looked upon with dismay by the majority of computer-chess researchers who had pursued methods that leveraged human understanding of the special structure of chess. When a simpler, search-based approach with special hardware and software proved vastly more effective, these human-knowledge-based chess researchers were not good losers. They said that ''brute force'' search may have won this time, but it was not a general strategy, and anyway it was not how people played chess. These researchers wanted methods based on human input to win and were disappointed when they did not.

A similar pattern of research progress was seen in computer Go, only delayed by a further 20 years. Enormous initial efforts went into avoiding search by taking advantage of human knowledge, or of the special features of the game, but all those efforts proved irrelevant, or worse, once search was applied effectively at scale. Also important was the use of learning by self play to learn a value function (as it was in many other games and even in chess, although learning did not play a big role in the 1997 program that first beat a world champion). Learning by self play, and learning in general, is like search in that it enables massive computation to be brought to bear. Search and learning are the two most important classes of techniques for utilizing massive amounts of computation in AI research. In computer Go, as in computer chess, researchers' initial effort was directed towards utilizing human understanding (so that less search was needed) and only much later was much greater success had by embracing search and learning.

In speech recognition, there was an early competition, sponsored by DARPA, in the 1970s. Entrants included a host of special methods that took advantage of human knowledge---knowledge of words, of phonemes, of the human vocal tract, etc. On the other side were newer methods that were more statistical in nature and did much more computation, based on hidden Markov models (HMMs). Again, the statistical methods won out over the human-knowledge-based methods. This led to a major change in all of natural language processing, gradually over decades, where statistics and computation came to dominate the field. The recent rise of deep learning in speech recognition is the most recent step in this consistent direction. Deep learning methods rely even less on human knowledge, and use even more computation, together with learning on huge training sets, to produce dramatically better speech recognition systems. As in the games, researchers always tried to make systems that worked the way the researchers thought their own minds worked---they tried to put that knowledge in their systems---but it proved ultimately counterproductive, and a colossal waste of researcher's time, when, through Moore's law, massive computation became available and a means was found to put it to good use.

In computer vision, there has been a similar pattern. Early methods conceived of vision as searching for edges, or generalized cylinders, or in terms of SIFT features. But today all this is discarded. Modern deep-learning neural networks use only the notions of convolution and certain kinds of invariances, and perform much better.

This is a big lesson. As a field, we still have not thoroughly learned it, as we are continuing to make the same kind of mistakes. To see this, and to effectively resist it, we have to understand the appeal of these mistakes. We have to learn the bitter lesson that building in how we think we think does not work in the long run. The bitter lesson is based on the historical observations that 1) AI researchers have often tried to build knowledge into their agents, 2) this always helps in the short term, and is personally satisfying to the researcher, but 3) in the long run it plateaus and even inhibits further progress, and 4) breakthrough progress eventually arrives by an opposing approach based on scaling computation by search and learning. The eventual success is tinged with bitterness, and often incompletely digested, because it is success over a favored, human-centric approach.

One thing that should be learned from the bitter lesson is the great power of general purpose methods, of methods that continue to scale with increased computation even as the available computation becomes very great. The two methods that seem to scale arbitrarily in this way are search and learning.

The second general point to be learned from the bitter lesson is that the actual contents of minds are tremendously, irredeemably complex; we should stop trying to find simple ways to think about the contents of minds, such as simple ways to think about space, objects, multiple agents, or symmetries. All these are part of the arbitrary, intrinsically-complex, outside world. They are not what should be built in, as their complexity is endless; instead we should build in only the meta-methods that can find and capture this arbitrary complexity. Essential to these methods is that they can find good approximations, but the search for them should be by our methods, not by us. We want AI agents that can discover like we can, not which contain what we have discovered. Building in our discoveries only makes it harder to see how the discovering process can be done.

Fig. 1. Sutton's bitter lesson, reprinted verbatim [1]. Blue: the essay's four motivating examples. Each comes from a domain with a fast, cheap, reliable feedback oracle. §III argues that much of SE analytics has no such oracle.

teach on personal hardware; defense, healthcare, finance, and law restrict what may leave the building; air-gapped and on-device settings have no cloud at all. For all of these, easier AI is the entry ticket.

SE itself has been teaching this lesson for decades. Modern software is mostly assembled from pre-made parts, and more reuse means more attack surface: left-pad, MOVEit ($12 billion), and similar supply-chain failures cost an estimated $24 billion in 2025 alone. Survivability argues the same way; SQLite ships its own tiny B-tree rather than adopting Berkeley DB (which Oracle later acquired and relicensed), and now runs on Mars. As SQLite's author puts it, "if you want to be free, that means doing things yourself" [16]. More code is more to maintain and more to go wrong. Even our configuration files are bloated: Xu et al. found that most users cannot cope with most of the options we give them [17].

There is also a supply problem. The big-data era ran on a silent premise: more data exists. Villalobos et al. project that the stock of human-generated public text runs out within years [18]. Synthetic replacements trade away privacy or fidelity [19], and training recursively on machine output collapses model quality [20].

Finally, more was never always better. For agile effort estimation, a support vector machine over TF-IDF features out-predicts deep learners while running 100 times faster [21]. For text mining StackOverflow, tuned simple learners ran 500 times faster than deep learning, at no loss [22]. Tree ensembles still beat deep networks on typical tabular data [23]. Often,

TABLE I
IN SE, FOR $y = f(x)$, ACQUIRING FEATURES x IS OFTEN FAR CHEAPER THAN DETERMINING LABELS y.

cheap feature acquisition (x)	**costly label determination (y)**
mine GitHub metadata (code size, dependencies)	estimate market value or total development effort
count system classes	audit actual maintenance effort
list design option permutations	validate configurations via stakeholder consensus
enumerate software parameters	execute full test suites for every unique build
identify learner hyper-parameters	run exhaustive grid searches
generate fuzzing inputs	execute tests, audit outputs

```
win     n     Lbs-    Acc+    Mpg+
  8    43   2334.2    16.3    27.4
 41    26   2038.7    17.1    30.0  Volume <= 111
 71    10   1871.7    18.1    32.0  |  Volume <= 83
 85     4   1831.7    19.0    32.5  |  |  Model <= 73
 62     6   1898.3    17.4    31.7  |  |  Model > 73
 22    16   2143.0    16.5    28.7  |  Volume > 83
 36     7   2098.1    18.0    28.6  |  |  Model <= 73
 12     9   2177.9    15.3    28.9  |  |  Model > 73
 26     4   2210.0    15.7    30.0  |  |  |  Volume > 97
  0     5   2152.2    15.0    28.0  |  |  |  Volume <= 97
-41    17   2786.1    15.1    23.5  Volume > 111
 -9     5   2279.8    14.6    28.0  |  Volume <= 116
-54    12   2997.1    15.3    21.7  |  Volume > 116
-44     8   2921.1    15.8    22.5  |  |  Model > 73
-38     4   2563.0    14.0    25.0  |  |  |  Clndrs <= 4
-50     4   3279.2    17.4    20.0  |  |  |  Clndrs > 4
-75     4   3149.0    14.4    20.0  |  |  Model <= 73
```

Fig. 2. An EZR tree for a three-goal automotive task: minimize weight (Lbs), maximize acceleration (Acc) and miles per gallon (Mpg). Each line is one node: win score, row count n, mean goal values, and the branch test. Built from 43 labels; the whole model fits in 17 lines and mentions three attributes.

better data or better tuning beats better (bigger) learners [24], [25], [26]. Hooker calls this the hardware lottery: ideas win because the available hardware favors them, not because they are best [27]. Big AI won a lottery. That is not the same as winning the argument.

Summing up: the bitter lesson has preconditions (a clear objective, abundant data, ever-cheaper compute). Chess and vision met all three; much of SE meets none. Where the preconditions fail, acting as if the lesson still applied is not rigor. It is habit.

III. HOW: ACTIVE LEARNING WITH EZR

Less is a very old idea. Around 1300, William of Ockham warned against multiplying entities beyond necessity. Pearson reduced data to a few principal components in 1901 [28]. Chang shrank nearest-neighbor classifiers to a few prototypes in 1974 [29]. Kohavi and John pruned feature sets in 1997 [30]. Active learning [31] and model distillation continue the tradition. Of these, we focus on active learning, since labels are what SE data lacks. As Table I shows, for $y = f(x)$ problems in SE, the independent features x come cheap while the dependent labels y come dear: measuring runtimes, running test suites, or eliciting expert judgment is slow and expensive, while unlabeled examples are nearly free.

Nor do the usual shortcuts fix this. Expert labeling is sluggish and fatigue-prone, taking hours per case for security or technical-debt judgments. Labels lifted from historical logs can mislead; in one audit, most logged technical-debt entries were false positives [32]. And asking an LLM to do the labeling does not close the gap, since human-model agreement is no proxy for ground truth [33]. Whatever the labeling method, budgets stay small. So the research question becomes: how to spend a small budget well?

The loop is short. Label a few random examples. Build a small model. Use that model to pick the most informative unlabeled example: explore where the model is most uncertain, exploit where it predicts the best outcomes, and shift slowly from explore to exploit. Label it, rebuild, repeat until the budget runs out. State-of-the-art optimizers such as SMAC3 run the same loop over an ensemble of decision trees [34], and earlier SE work did so over pairs of performance models [35]. But note the hidden cost in the heavyweight versions: every new label triggers a rebuild of the whole ensemble, so the machinery around the labels can cost far more than the labels themselves. The EZR toolkit strips the loop to a few hundred lines of dependency-free Python: a Naive Bayes-style learner picks the labels, then a small regression tree summarizes them [4]. In that loop, labeling is the only expensive step; everything else runs in milliseconds.

Testing such claims needs many tasks, not one case study. The MOOT repository holds 127 multi-objective optimization datasets extracted from recent SE papers [36]. The tasks cover software configuration, performance tuning, product lines, project health, defect prediction, testing, cost estimation, cross-domain generalization, and text mining. They range from 3 to 1,044 independent attributes, one to eight goals, and 93 to 100,000 rows. Performance is scored as $win = 100 \times (1 - regret)$, where regret is the distance from the best result in the full dataset, normalized so that 100 means best-known and 0 means as bad as random guessing.

This is best shown by example. In Figure 2, rows describe cars; the goals are to minimize weight while maximizing acceleration and miles per gallon. From 43 labels, the root scores $win = 8$: barely better than guessing. Two tests down (engine volume at most 83, model year at most '73) sit four cars scoring $win = 85$: light, quick, thrifty. The worst leaf ($win = -75$) holds heavy gas-guzzlers, so the tree also says, just as plainly, what to avoid. The whole policy mentions three attributes; a domain expert can dispute it line by line. To apply it, sort new unlabeled examples down its branches and spend the labeling budget only on those landing in the best leaves.

Does it work at scale? A study of 100,000 runs over MOOT proceeded as follows [3]. Pick one of the 127 datasets. Hide the labels; split off a holdout. Let the active learner spend at most 50 labels on the training side, then build a tree from what it learned. Sort the holdout through that tree and label only the first five suggestions, scoring the best of those five. Median result across all runs: 85% of the best known. The whole study ran in two hours on one laptop.

How does that compare? In a tournament run at matched budgets, NSGA-II [37] needed roughly 1,000 evaluations to

reach what these methods reach in 50 [38], and EZR's per-task runtimes sit in the milliseconds. That is not just faster science; 500 times less CPU is 500 times less energy, every run. Nor is this the first such warning [39], [40].

Are the resulting models small enough to trust? Across the 127 datasets, EZR's trees use four to nine attributes, out of as many as 1,044 [5]. Systems described by a thousand variables are routinely controlled by fewer than ten; effort spent modeling the rest buys noise. Small is only useful if nothing important got thrown away, so that work also ran a harder test: if EZR's tree uses N attributes, ask state-of-the-art attribute selectors for their top N, build models from each selection, and compare. Trees built from EZR's few attributes usually tied with, or came close to, the best of those selectors [5]. A model that fits on half a page can be checked by a domain expert before lunch. None of the black-box alternatives offer that.

IV. Why Not More Easier AI?

If the evidence is this good, why is easier AI so rare? We offer four answers.

First, weak empirical standards. LLM papers now flood SE venues [41], yet in a survey of 229 of them, only 5% compared against a simpler approach [4]. That is an error, not a style choice, since simpler methods can be better, faster, or both [26], [22], [21]. The fix is old: Cohen's 1995 empirical-AI text recommended baselining against straw-man ablations [42]. So, before shipping a clever method, build the stupidest straw man that could possibly work. Surprisingly often, the straw man wins, and the clever method should be thrown away [40].

Second, incentives. Big sells. A data center has a ribbon the mayor can cut. A 300-line script has no launch event, no special hardware to sell (it is too fast to need any), and no valuation story. The spectacle funds itself; the simplification does not.

Third, the simplicity paradox. Simplicity must be earned twice. Earned in experience, since knowing what to cut takes years of mistakes ("experience is the name every one gives to their mistakes," as Wilde had it). And earned in compute, since certifying that simple suffices means running both the simple and the complex method at scale; the tournament behind §III consumed some 20 months of CPU [38]. Simple results are expensive to prove and cheap-looking once proved. A bad bargain for a career, and researchers know it.

Fourth, our brains fight us. In studies covering hundreds of subjects, Adams et al. found that people systematically reach for additive changes over subtractive ones, by roughly four to one [43]. We suspect the simpler methods reported here were not found earlier for the simplest of reasons: nobody was looking down.

Call the sum of these the inverse bitter lesson. Sutton warned that researchers cling to clever hand-crafted methods when scale would do better [1]. The companion failure is just as real: reaching for scale reflexively, an LLM and an embedding API for a problem that thirty lines of Naive Bayes over 1,700 rows would solve. The first failure wastes a project. The second, repeated across 95% of a literature [4], wastes a field.

V. Future Plans

The emerging result: 50 labels often suffice. The new idea: treat that as a research program, the data-light challenge, asking for which SE tasks a handful of labels suffices and when it fails [3]. A full-length result needs four things.

Characterization. Not all tasks simplify; we need to predict which. The plan: regress label-budget requirements against measurable task features (dimensionality, goal count, noise, intrinsic dimension) across an expanded MOOT [36] that adds failure cases, since a corpus that simple methods always win teaches nothing.

Tournaments. The comparison set must include what people actually use: state-of-the-art optimizers [34], [37] and LLM-based agents, run head-to-head at matched labeling budgets [38], with energy reported alongside accuracy.

Hybrids. Easier AI need not be anti-LLM. Early results suggest LLMs can warm-start active learning with a few good initial guesses [6], and that running classical optimizers before LLMs beats the reverse order [7]. The open question is where the crossover sits: how few labels before the prior knowledge inside an LLM stops paying rent?

Standards and teaching. We will draft a one-line addition to SE review checklists ("compared to what simpler?") and an easier-AI-first curriculum, where students meet the 300-line version before the trillion-parameter one.

What could kill this idea? Three things, all testable. MOOT could be biased toward tasks where simple methods win (fix: grow the corpus adversarially). Safety-critical work may reject 85% of best (the characterization study should mark that boundary). And the evidence here is tabular; whether it extends to code, traces, and text is what the hybrid studies will answer. Either way, the community gets what it lacks: an evidence-based boundary between easy and hard.

VI. Conclusion

Lehman told us that complexity grows unless work is done to reduce it [2]. This paper argues we must (economics, energy, trust, access) and that we can (50 labels, four to nine attributes, 85% of best, one laptop). The tools and data are public [4], [36]. Sutton was right where feedback is free. The sweeter lesson covers the rest: where labels are dear, a few dozen well-chosen ones go most of the way. Baseline the simple. At review time, ask "compared to what simpler?" Teach the easy version first. Scale only when the simple fails.

References

[1] R. S. Sutton, "The bitter lesson," http://www.incompleteideas.net/IncIdeas/BitterLesson.html, 2019.

[2] M. M. Lehman, "Programs, life cycles, and laws of software evolution," *Proceedings of the IEEE*, vol. 68, no. 9, pp. 1060–1076, 1980.

[3] K. K. Ganguly and T. Menzies, "How low can you go? The data-light SE challenge," *Proceedings of the ACM on Software Engineering*, vol. 3, no. FSE, p. Article FSE185, 2026.

[4] T. Menzies and S. Srinivasan, "Can AI be easy? Lessons learned from the EZR.py toolkit," *arXiv preprint arXiv:2606.03640*, 2026.

[5] A. Rayegan and T. Menzies, "Minimal data, maximum clarity: A heuristic for explaining optimization," *Journal of Systems and Software*, vol. 238, p. 112897, 2026.

[6] L. Senthilkumar and T. Menzies, "Can large language models improve SE active learning via warm-starts?" *ACM Transactions on Software Engineering and Methodology*, 2026.

[7] S. Srinivasan and T. Menzies, "Better together, in the right order: Classical-then-LLM optimization for SE," *arXiv preprint*, 2026.

[8] N. Wirth, "A plea for lean software," *IEEE Computer*, vol. 28, no. 2, pp. 64–68, 1995.

[9] Bain & Company, "Global technology report 2025," Bain & Company, Tech. Rep., 2025.

[10] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

[11] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?" in *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021, pp. 610–623.

[12] S. Kabir, D. N. Udo-Imeh, B. Kou, and T. Zhang, "Is stack overflow obsolete? An empirical study of the characteristics of ChatGPT answers to stack overflow questions," in *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI)*, 2024.

[13] S. Baltes, M. Cheong, and C. Treude, ""an endless stream of AI slop": The growing burden of AI-assisted software development," arXiv preprint arXiv:2603.27249, 2026.

[14] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.

[15] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 3645–3650.

[16] D. R. Hipp, "The untold story of SQLite," Interview, CoRecursive podcast #66, https://corecursive.com/066-sqlite-with-richard-hipp/, 2021.

[17] T. Xu, L. Jin, X. Fan, Y. Zhou, S. Pasupathy, and R. Talwadker, "Hey, you have given me too many knobs! Understanding and dealing with over-designed configuration in system software," in *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2015, pp. 307–319.

[18] P. Villalobos, A. Ho, J. Sevilla, T. Besiroglu, L. Heim, and M. Hobbhahn, "Will we run out of data? Limits of LLM scaling based on human-generated data," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[19] X. Ling, T. Menzies, C. Hazard, J. Shu, and J. Beel, "Trading off scalability, privacy, and performance in data synthesis," *IEEE Access*, vol. 12, pp. 26 642–26 654, 2024.

[20] I. Shumailov, Z. Shumaylov, Y. Zhao, Y. Gal, N. Papernot, and R. Anderson, "AI models collapse when trained on recursively generated data," *Nature*, vol. 631, no. 8022, pp. 755–759, 2024.

[21] V. Tawosi, R. Moussa, and F. Sarro, "Agile effort estimation: Have we solved the problem yet? Insights from a replication study," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 2677–2697, 2023.

[22] S. Majumder, N. Balaji, K. Brey, W. Fu, and T. Menzies, "500+ times faster than deep learning: A case study exploring faster methods for text mining StackOverflow," in *Proceedings of the 15th International Conference on Mining Software Repositories (MSR)*, 2018, pp. 554–563.

[23] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Proceedings of NeurIPS, Datasets and Benchmarks Track*, 2022.

[24] A. Agrawal and T. Menzies, "Is "better data" better than "better data miners"?" in *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, 2018, pp. 1050–1061.

[25] W. Fu, T. Menzies, and X. Shen, "Tuning for software analytics: Is it really necessary?" *Information and Software Technology*, vol. 76, pp. 135–146, 2016.

[26] W. Fu and T. Menzies, "Easy over hard: A case study on deep learning," in *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2017, pp. 49–60.

[27] S. Hooker, "The hardware lottery," *Communications of the ACM*, vol. 64, no. 12, pp. 58–65, 2021.

[28] K. Pearson, "On lines and planes of closest fit to systems of points in space," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 2, no. 11, pp. 559–572, 1901.

[29] C.-L. Chang, "Finding prototypes for nearest neighbor classifiers," *IEEE Transactions on Computers*, vol. C-23, no. 11, pp. 1179–1184, 1974.

[30] R. Kohavi and G. H. John, "Wrappers for feature subset selection," *Artificial Intelligence*, vol. 97, no. 1-2, pp. 273–324, 1997.

[31] B. Settles, "Active learning literature survey," University of Wisconsin-Madison, Tech. Rep. Computer Sciences Technical Report 1648, 2009.

[32] Z. Yu, F. M. Fahid, H. Tu, and T. Menzies, "Identifying self-admitted technical debts with jitterbug: A two-step approach," *IEEE Transactions on Software Engineering*, vol. 48, no. 5, pp. 1676–1691, 2022.

[33] T. Ahmed, P. Devanbu, C. Treude, and M. Pradel, "Can LLMs replace manual annotation of software engineering artifacts?" in *Proceedings of the International Conference on Mining Software Repositories (MSR)*, 2025.

[34] M. Lindauer, K. Eggensperger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, and F. Hutter, "SMAC3: A versatile bayesian optimization package for hyperparameter optimization," *Journal of Machine Learning Research*, vol. 23, no. 54, pp. 1–9, 2022.

[35] V. Nair, Z. Yu, T. Menzies, N. Siegmund, and S. Apel, "Finding faster configurations using FLASH," *IEEE Transactions on Software Engineering*, vol. 46, no. 7, pp. 794–811, 2020.

[36] T. Menzies, K. K. Ganguly, A. Rayegan, and S. Srinivasan, "MOOT: A repository of many multi-objective optimization tasks," in *Proceedings of the International Conference on Mining Software Repositories (MSR)*, 2026.

[37] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[38] K. K. Ganguly and T. Menzies, "Which optimizer, at what budget? A tournament of optimizers for search-based SE," 2026, under review.

[39] J. Chen, V. Nair, and T. Menzies, ""Sampling" as a baseline optimizer for search-based software engineering," *IEEE Transactions on Software Engineering*, vol. 45, no. 6, pp. 597–614, 2019.

[40] T. Menzies, "Shockingly simple: "Keys" for better AI for SE," *IEEE Software*, vol. 38, no. 2, pp. 114–118, 2021.

[41] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.

[42] P. R. Cohen, *Empirical Methods for Artificial Intelligence*. Cambridge, MA: MIT Press, 1995.

[43] G. S. Adams, B. A. Converse, A. H. Hales, and L. E. Klotz, "People systematically overlook subtractive changes," *Nature*, vol. 592, pp. 258–261, 2021.